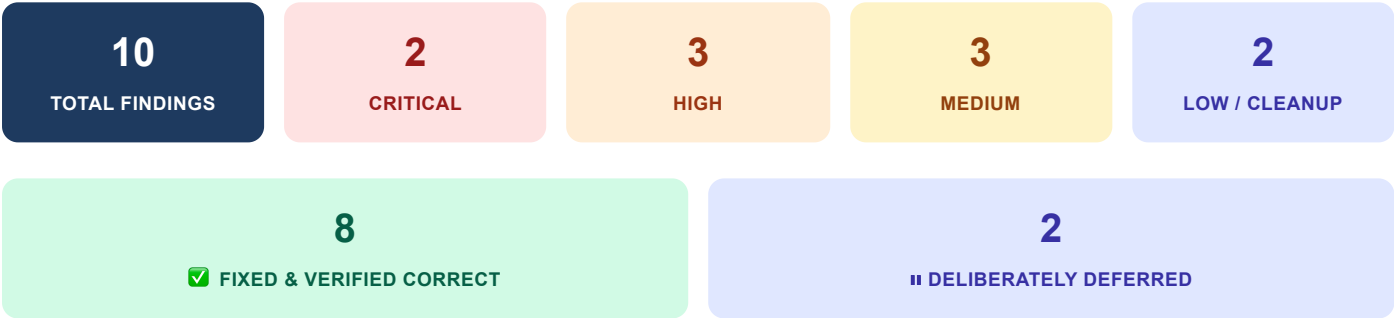


PR #64 — PMO PRD Gap Closure Phase 1 + AI Governance + Alert Role-Scoping

Repo: ehiddenbrain/scale · Branch: integration/pmo-alerts-phase2 → main · Status: Open

This is a plain-language write-up of a code review done on PR #64 — every finding below was verified directly against the real PR code (not the PR description), tracing the exact execution path a real user or a real chat question would hit. Each item below explains **what's happening**, in plain terms; **why it matters** (a concrete scenario, not just "this could be a bug"); and a **suggested fix** — sized to be a small, targeted change, not a rewrite.

Update — 2026-09-13: Veeren pushed a fix commit ([630bee5](#)) addressing 8 of the 10 findings below, plus two PR comments explaining why the remaining 2 are deliberately deferred rather than missed. Every fix was re-verified directly against the new diff, not taken on the strength of the commit message — status is marked on each finding below. Net result: **8 fixed and confirmed correct, 2 deferred with sound reasoning** (verified independently, not just accepted).



● Critical — silently wrong behavior, no error shown to anyone

FINDING 1

CRITICAL

A failed AI call disappears completely — no log, no quota deduction

scale_project_management/chat.py — around line 991

Fix status: ✅ Fixed and verified correct. `frappe.db.commit()` added immediately after `record_turn()`, and after every failure-path `record_call()` — including `ai_utils.call_llm()`'s single-shot site, which had the identical gap but wasn't explicitly named in the original review. Confirmed by reading the diff directly.

One residual worth a follow-up, not a re-open: the new shared helper's *success* path still has no explicit commit. In a multi-round chat turn where round 1 succeeds and round 2 later fails, round 1's own log row can still be erased by the same rollback mechanism — narrower than the original bug, but the same root cause survives in that one corner.

WHAT'S HAPPENING

This PR adds two things this exact scenario depends on: an **AI Call Log** that's supposed to record every single provider call (including failed ones, so nothing is invisible), and a **daily message quota** per user that's supposed to still count against a question even if it crashes partway (otherwise a broken retry loop could burn unlimited provider spend for free — this is explicitly called out as the reasoning in the code's own comments).

Both of these are written to the database with a plain `.insert()` and no explicit `frappe.db.commit()`. That's normally fine — Frappe automatically commits everything at the end of a successful request. But when the AI provider call itself fails (timeout, 5xx, network error), the code logs the failure and then re-raises the exception, on purpose, so the error still bubbles up. The problem: nothing in `ask()` catches that exception anywhere above this point. Frappe's own request handler sees an unhandled exception and — this is standard, checked directly in Frappe's own code (`frappe/app.py`) — **rolls back the entire database transaction for that request**. That rollback erases both the "quota consumed" row and the "call failed" log row that were just written, because they were never committed independently.

WHY IT MATTERS

Picture a provider timeout (these happen — proxies, rate limits, network blips). Right now: the AI Call Log shows nothing happened, and the user's daily quota is untouched — as if the question was never asked. A user (or a script) that automatically retries on failure can keep asking the same question forever with zero quota cost and zero record of it ever failing. This directly undermines the two features this exact PR is adding (cost visibility and quota enforcement) in the one scenario — a real provider failure — where they matter most.

SUGGESTED FIX

Add an explicit `frappe.db.commit()` right after each of these two writes, so they survive even if something later in the same request fails:

```
_log_name = ai_limits.record_turn(...)
+ frappe.db.commit()

...

except Exception as _exc:
    record_call(..., error=str(_exc))
    + frappe.db.commit()
    raise
```

This is a small, localized change — two extra lines — and it doesn't change any other behavior. It just makes sure these two specific "audit trail" writes are durable the moment they happen, independent of whatever happens next in the request.

FINDING 2

The new "who has this skill?" chatbot tool confidently says "nobody" for roles who are actually allowed to ask it

CRITICAL

scale_project_management/resource_skill_matching.py — line 53 (Employee lookup)

Fix status: || Deliberately deferred, not fixed in this commit. Veeren's PR comment: this is a known, pre-existing HR/PMO data-ownership boundary, not an oversight — `Employee Skill Map` / `Skill` are HR-owned and deliberately not opened to PMO roles pending a real access decision, which needs a product call rather than a quiet DocPerm change bundled into this PR. That's a legitimate way to handle an ownership question. It doesn't change the underlying fact, though: the tool still gives a confidently wrong "nobody qualified" answer today, regardless of whose call it is to fix. Worth tracking as an explicit follow-up rather than letting it go quiet.

WHAT'S HAPPENING

This PR adds a chat tool, `get_task_skill_matches`, so a PMO user can ask "who has the skill needed for this task?" It's wired to be usable by **Department Head, Accounts Manager, Projects Manager, and PMO Director** — that's the access level the tool is registered at in `chat.py`.

But the actual lookup for Employee skills reads from two doctypes — `Employee Skill Map` and `Skill` — and neither one grants read access to any of those four roles. Checked directly against the real permission definitions:

- **Employee Skill Map**: only `System Manager` can read it.
- **Skill**: only `System Manager` and `HR Manager` can read it.

The lookup function (`readable_list()`) is written to catch that permission error and quietly return an empty list instead of showing an error — which is a deliberate, reasonable design for a chat surface (nobody wants a chatbot to spit out a stack trace). But it means the "no permission" case and the "genuinely nobody has this skill" case look *exactly the same* to the person asking.

WHY IT MATTERS

A PMO Director — one of the roles this feature is explicitly built for — asks the chatbot "who can do this task?" The tool call technically succeeds. Every real Employee candidate silently gets filtered out by the permission check, and the assistant confidently answers "nobody has this skill" — even when three qualified employees are sitting right there in the system. There's no error, no warning, nothing to suggest the answer is wrong. This is the worst kind of bug for an AI feature: a wrong answer delivered with full confidence. It's also untested — the test file (`test_skill_matching.py`) only ever runs as an admin-level user, so this exact gap wouldn't show up in the test suite.

SUGGESTED FIX

Grant read access on `Employee Skill Map` (and confirm `Skill` too, since HR Manager already has it but the other three roles don't) to whichever role actually represents "can use PMO chat features" in this app — likely the same role or role-set already used for other PMO-only data, or a new Custom DocPerm row for Department Head / Accounts Manager / Projects Manager / PMO Director directly. Then add one test to `test_skill_matching.py` that runs the tool as *one of these roles* (not as Administrator) with real seeded skill data, and asserts the result is non-empty — that's the test that would have caught this before merge, and it'll catch the next version of this same class of bug too.

● High — real, demonstrable defects

FINDING 3

HIGH

The budget forecast dashboard now sorts by the wrong column

`scale_project_management/fixtures/report.json - "PMO Forecast Cost Dashboard" query`

Fix status:  **Fixed and verified correct.** `order by 5 desc` → `order by 8 desc`, confirmed directly in the diff — exactly the one-character fix suggested below.

WHAT'S HAPPENING

This report used to have 6 columns, with the last one — "Forecast vs Planned" (the overrun percentage) — at position 6, and the query correctly said `order by 6 desc` so the worst overruns sort to the top. This PR adds two new columns ("Best Case" and "Worst Case" burn estimates) *before* that percent column, which pushes it from position 6 to position 8. The `order by` was changed too — but to `5`, not `8`. Confirmed by comparing this PR's query against the version currently on `main` line by line — this is a real, checked-in regression, not a guess.

BEFORE (ON MAIN) VS. AFTER (THIS PR)

```
# main - 6 columns, percent column at position 6
... "Forecast Final Cost:Currency:150",
    case when ... end as "Forecast vs Planned:Percent:150"
from ...
order by 6 desc    ✓ correct - sorts by the percent column

# PR #64 - 8 columns, percent column now at position 8
... "Forecast Final Cost (Expected):Currency:180",
... "Best Case (-15% Burn):Currency:180",
... "Worst Case (+15% Burn):Currency:180",
    case when ... end as "Forecast vs Planned:Percent:150"
from ...
order by 5 desc    ✗ now sorts by "Forecast Final Cost (Expected)" - an absolute ₹ amount
```

WHY IT MATTERS

Column 5 is "Forecast Final Cost (Expected)" — a raw currency amount, not a percentage. So the dashboard now sorts by *how big the project's forecast cost is*, not *how badly it's overrunning*. A large, perfectly healthy ₹2 crore project sits at the top just because it's expensive, while a small ₹5 lakh project that's forecasting a genuine 300% budget blowout sinks to the bottom of the list — exactly the project a PMO Director most needs to see first.

SUGGESTED FIX

One-character fix — change `order by 5 desc` to `order by 8 desc` in that query, so it sorts by the actual "Forecast vs Planned" percent column at its new position.

FINDING 4

HIGH

A malformed (but "successful") AI response crashes the chat with zero log entry`chat.py` line ~1001 (tool round), ~1091 (forced final answer), `ai_utils.py` `call_llm()` line ~110

Fix status: ✅ **Fixed and verified correct.** Folded into the same commit as Finding 9's fix, as suggested — `resp.json()` and the message-extraction indexing now sit inside the try/except in the new shared `post_and_log()` helper, at all three call sites. Confirmed by reading the helper's source directly.

WHAT'S HAPPENING

The AI Call Log guarantee this PR adds is "every failed call gets a row." The code correctly wraps the network call itself (`requests.post` + `raise_for_status()`) in a try/except that logs on failure. But right after that try/except block ends, the very next lines — `resp.json()` , then reading `["choices"][0]["message"]` out of it — sit *outside* that protection. This exact pattern is repeated at three separate places in the codebase.

THE GAP, VISUALLY

```
try:
    resp = requests.post(...)
    resp.raise_for_status()
except Exception as _exc:
    record_call(..., error=str(_exc))    # ✓ logged if the HTTP call itself fails
    raise

resp_json = resp.json()                # ✗ NOT protected
reply = resp_json["choices"][0]...      # ✗ NOT protected
```

WHY IT MATTERS

An HTTP 200 with a truncated, empty, or unexpectedly-shaped body (a flaky proxy, a provider having a bad day, a rate-limit response that still returns 200) doesn't fail the way the code expects — it fails one line later, as an unhandled crash with, ironically, *no log entry at all* — the one failure mode the logging is supposed to catch, slipping through the exact gap next to the code that catches everything else.

SUGGESTED FIX

Move `resp.json()` and the indexing that pulls the message out of it *inside* the same try block, at all three locations. The except block already knows how to log and re-raise — it just needs to also see this failure mode. (This pairs naturally with Finding 9 below, since fixing it in one shared helper fixes it at all three sites at once.)

FINDING 7

HIGH

"AI governance across all provider call sites" is only true for PMO`ai_call_log_hooks.py, ai_limits.py – vs. scale_finance / scale_hr / scale_sales chat.py`

Fix status: **II Deliberately deferred, not fixed in this commit — and the reasoning checks out.** Veeren's PR comment: the real fix means relocating `ai_call_log_hooks` / `ai_limits` into the shared `scale` app, since Finance/HR/Sales don't declare `scale_project_management` as a dependency. **Verified independently:** checked `required_apps` in all four apps' `hooks.py` directly — Finance/HR/Sales each declare `scale` (not `scale_project_management`), so importing PMO's own logging/quota code as-is would be an undeclared cross-app dependency that only happens to work because everything is installed together on this bench. This is the architecturally correct call, not an excuse.

WHAT'S HAPPENING

This PR's own description says the AI Call Log tracks calls "across all provider call sites" and adds daily quota governance. Checked directly: `scale_finance/chat.py`, `scale_hr/chat.py`, and `scale_sales/chat.py` — all three touched in this same PR (for the Sonnet→Haiku model switch) — have **zero** references to `record_call`, `ai_limits`, `record_turn`, or anything from the new AI Call Log doctype. Only PMO's own `chat.py` and `ai_utils.py` actually call into this new logging/quota system.

WHY IT MATTERS

Two consequences: (1) the new **AI Spend** page — the whole point of which is org-wide AI cost visibility for the PMO Director — is structurally undercounting. Three of the four chatbot surfaces (Finance, HR, Sales) are completely invisible to it. (2) The daily message quota, presented as a real cost-control mechanism, only actually applies to PMO users. A Finance or HR or Sales chatbot user has no cap at all.

SUGGESTED FIX

Two options, either is reasonable depending on scope for this PR: (a) narrow the PR description to say this phase covers PMO only, with Finance/HR/Sales as a named, tracked follow-up — cheapest, and honest about what's actually shipped; or (b) wire the same `record_call` /quota calls into the other three apps' `chat.py` / `ai_utils.py` the same way PMO's already are — more work, but closes the gap for real. Given all three already got touched in this PR for the model flip, this might be a small, well-scoped fast-follow rather than a big lift.

 **Medium — real edge cases and reliability risks**

FINDING 5**MEDIUM****Asking about a project that doesn't exist reports it as perfectly healthy**`scale_project_management/project_health_score.py – compute_project_health_score(), line 116`

Fix status: ✅ **Fixed and verified correct.** `if not frappe.db.exists("Project", project): frappe.throw(...)` added at the top of the function — exactly the code suggested below, confirmed in the diff.

WHAT'S HAPPENING

The new AI Project Health Score never checks whether the project name it's given actually exists. Each of its four scoring dimensions (schedule, budget, risk, milestones) queries other doctypes filtered by that project name — and for a name that doesn't exist, every one of those queries correctly returns "nothing found," which this scoring logic treats as "no problems in this dimension" and scores as a clean 100.

WHY IT MATTERS

Ask the chatbot for the health score of a misspelled or deleted project name (e.g. a typo like "PROJ-0100" instead of "PROJ-0010", or a project since deleted) and instead of "I couldn't find that project," it reports a perfect or near-perfect 100/100 health score — a confidently wrong answer about a project that isn't even real. No test in the PR covers this input.

SUGGESTED FIX

Add one check at the top of `compute_project_health_score()` :

```
if not frappe.db.exists("Project", project):
    frappe.throw(_("Project {0} not found").format(project))
```

FINDING 6

MEDIUM

Two chat questions sent at almost the same moment can both slip past the daily quota

scale_project_management/ai_limits.py – check_limit() / record_turn(), line 97

Fix status: ✅ **Fixed and verified correct.** `check_limit()` now runs `SELECT COUNT(*) ... FOR UPDATE` instead of a plain `frappe.db.count()`, taking a row/gap lock a concurrent request for the same user has to wait behind — the standard, correct pattern for this race, and it pairs correctly with Finding 1's fix (the request now commits immediately after `record_turn()`, so the lock's protection isn't undone by a delayed commit).

WHAT'S HAPPENING

`check_limit()` counts how many messages a user has already sent today with a plain database count — no row locking. `record_turn()` is a separate insert that happens afterward. Between those two steps, there's a small window where a second, near-simultaneous request can run the exact same count-check before either request's insert has landed — this is the classic "check-then-act" race condition.

WHY IT MATTERS

A user with exactly 1 message left opens two browser tabs (or has a flaky connection that silently double-sends) and asks a question from both at nearly the same instant. Both requests check the count before either has recorded its own usage, so both pass the check — the daily cap is quietly exceeded by one. Low real-world stakes on its own (worst case, one extra message for one user on one day), but worth fixing given this is explicitly positioned as a cost-control mechanism, not just a UX nudge.

SUGGESTED FIX

Make the count-then-insert sequence atomic — e.g. run the counting query with `frappe.db.sql(..., for_update=True)` so it takes a row lock other concurrent requests for the same user have to wait behind, before the insert happens. Given the volumes involved (one user's daily message count), this has no meaningful performance cost.

FINDING 8

MEDIUM

The just-fixed alert cap will silently break again the next time a trigger type is added

assistant_tools/get_portfolio_alerts.py — MAX_PORTFOLIO_ALERTS, line 57

Fix status: ✅ Fixed and verified correct. `MAX_PORTFOLIO_ALERTS = max(20, len(TRIGGERS) + 3)`, with `TRIGGERS` now imported directly from `get_project_alerts` — exactly the fix suggested below, confirmed in the diff.

WHAT'S HAPPENING

This PR itself fixes a real, well-documented bug: `MAX_PORTFOLIO_ALERTS` was 10, but the app now has 17 different alert trigger types, so the last several trigger types could never appear in a portfolio-wide alert answer no matter how many real alerts of those types existed — confirmed live by the PR author before this fix. The fix raises the cap to 20. Good fix — but the number 20 is just typed in by hand. Nothing in the code ties it to the actual count of trigger types (17 right now), so the exact same silent failure will happen again the moment an 18th trigger type is added, with no warning that it's happened.

WHY IT MATTERS

This bug already happened once (that's why this PR fixes it) and the fix doesn't stop it from happening again — it just buys headroom for the current trigger count. The next engineer who adds trigger type #18 has no reason to know this number needs updating too, and won't get an error telling them.

SUGGESTED FIX

Tie the two together instead of maintaining them separately, e.g.:

```
from scale_project_management.assistant_tools.get_project_alerts import TRIGGERS
MAX_PORTFOLIO_ALERTS = max(20, len(TRIGGERS) + 3)
```

or, if that import creates an awkward dependency, at minimum add an assertion that fails loudly at import time if the cap ever falls behind the trigger count again:

```
assert MAX_PORTFOLIO_ALERTS >= len(TRIGGERS), "raise MAX_PORTFOLIO_ALERTS — see
get_portfolio_alerts.py"
```

● Low — cleanup / maintainability, not urgent

FINDING 9

LOW

The same ~15-line "call the provider and log it" block is copy-pasted three times

```
chat.py (tool round + forced final round), ai_utils.py call_llm()
```

Fix status:  **Fixed and verified correct.** Extracted into a new shared `ai_call_log_hooks.post_and_log()` helper, called from all three sites — exactly the fix suggested below, and it correctly folds in Finding 4's fix at the same time.

WHAT'S HAPPENING

The pattern "start a timer, call the provider, log success or failure, re-raise on failure" is hand-typed identically at three separate call sites instead of living in one shared function.

WHY IT MATTERS

Not a bug today, but a maintenance risk: a future 4th call site (or the Finance/HR/Sales wiring from Finding 7) means hand-copying this scaffold correctly a fourth time, and a small slip — logging in the wrong order, forgetting the re-raise — produces inconsistent log data at just that one spot, which is exactly the kind of bug that's hard to notice until someone's trying to reconcile AI spend numbers.

SUGGESTED FIX

Extract one small helper — something like `_post_and_log(endpoint, headers, payload, *, feature, call_type, turn_id, model)` — that does the try/post/raise_for_status/log/parse/return sequence once (including the `resp.json()` fix from Finding 4), and call it from all three sites.

FINDING 10

LOW

The skill-matching tool does far more database queries than it needs to

`resource_skill_matching.py – _employee_skill_candidates() / _contractor_skill_candidates(), line 49`

Fix status:  **Fixed and verified correct.** Both functions now do one bulk `frappe.get_all()` against the child doctype (`Employee Skill` / `PMO Resource Skill`), joined back via a dict — exactly the pattern suggested below, confirmed in the diff. N+1 eliminated.

WHAT'S HAPPENING

Both candidate-gathering functions first fetch a list of names (all Employee Skill Maps, or all active Contractor Profiles), then loop over that list calling `frappe.get_cached_doc()` once per name just to read that one document's own skill child-table. That's one query to get the list, plus one more query per row — a classic "N+1" pattern — instead of one bulk query against the child table directly.

WHY IT MATTERS

This tool is chat-reachable, meaning every time someone asks a skill-matching question, this runs synchronously inside that one request. On an organization with a few hundred employees and contractors, that's a few hundred extra individual queries for a question that only needed a handful of matching rows — adding real, avoidable latency to a chat answer.

SUGGESTED FIX

Replace the per-document loop with one bulk fetch against the child doctype, filtered by the parent names already collected, e.g.:

```
skill_rows = frappe.get_all(
    "Employee Skill",
    filters={"parent": ["in", [m["name"] for m in maps]]},
    fields=["parent", "skill", "proficiency"],
)
```

then join back to each Employee Skill Map's `employee` / `employee_name` using the list already fetched — same result, one query instead of N+1. The same change applies to the Contractor Profile side.

How this review was done: a multi-angle automated code review against the real PR #64 diff (fetched directly from GitHub, not the PR description), followed by manual line-by-line re-verification of every finding against the actual code before writing this report — including diffing the dashboard query against its current version on `main` to confirm Finding 3 precisely, and checking the real DocType permission definitions (not assumptions) for Finding 2. Every code snippet above is either an exact quote or a minimal, deliberately-simplified illustration of the real fix shape — not a copy-paste-ready patch, since the exact wording is yours to decide.

How the fix status above was verified: each fix was re-checked against the actual diff between the originally-reviewed commit (`8709fb7`) and the new head (`630bee5`) — not accepted on the strength of Veeren's own summary table or comments. The two deferrals' stated reasoning was independently spot-checked rather than taken at face value: Finding 7's cross-app dependency claim was confirmed directly against all four apps' `hooks.py` `required_apps` declarations. One small inconsistency noted for accuracy: Veeren's own PR comment says "7 of 10 findings fixed" but the table beneath it lists 8 rows — almost certainly a wording slip (10 – 2 deferred = 8, matching the table), not a technical discrepancy. Nothing in this document has been posted to GitHub or applied to any branch.